Отчетная практическая работа №1

«Основы контейнеризации: от разработки до публикации в GitHub Container Registry»

Цели работы:

1. Освоить базовые команды Docker для управления контейнерами, образами и томами
2. Научиться писать эффективные Dockerfile с использованием multi-stage сборки
3. Настроить CI/CD пайплайн в GitHub Actions для автоматической сборки образа
4. Опубликовать образ в GitHub Container Registry (GHCR)

Технологический стек

- **Хост ОС:** Любая ОС с поддержкой Docker
- **Среда контейнеризации:** Docker Desktop для Windows
- **Язык программирования:** Go (для backend части)
- **База данных:** PostgreSQL (в отдельном контейнере)
- **Веб-сервер:** Nginx (для раздачи статики)
- **Система контроля версий:** Git + GitHub
- **CI/CD:** GitHub Actions

Часть 1. Подготовка инфраструктуры

1.1 Установка Docker Desktop (для отчета не нужно!)

1. Скачайте Docker Desktop for Windows с официального сайта
2. При установке **обязательно** отметьте опцию "Use WSL 2 instead of Hyper-V" (если не отмечено)
3. После установки запустите Docker Desktop и убедитесь, что в статусе отображается "Engine running"
4. Откройте PowerShell и проверьте установку:

```
docker --version
docker info
```

1.2 Подготовка репозитория на GitHub (для отчета не нужно!)

1. Создайте новый публичный* репозиторий на GitHub с названием `containers-lab-1`
2. Склонируйте репозиторий локально:

```
git clone https://github.com/ВАШ_ЛОГИН/containers-lab-1.git
cd containers-lab-1
```

3. Создайте структуру директорий:

```
mkdir app, nginx, init-scripts, .github\workflows
```

* Важно! Не рекомендуется создавать публичный репозиторий с вашими персональными данными! Для практической работы можно использовать аббревиатуру из ФИО на латинице, указывать только номер группы без специальности и года! Полностью данные ФИО и группы нужно указать в файле отчета REPORT.md, который будет у вас локально, делать commit и push с ним в публичный репозиторий не нужно! Также в вставках с кодом тоже можно указать только аббревиатру на латинице и сокращенный номер группы без года и специальности!

Часть 2. Знакомство с командами Docker (для отчета нужно прикрепить ввод/вывод из терминала либо скриншоты)

2.1 Работа с образами

Задание 2.1.1: Выполните базовые операции с образами

```
# Поиск образов в Docker Hub
docker search nginx

# Скачивание образа
docker pull nginx:alpine


# Просмотр локальных образов
docker images

# Просмотр истории слоев образа
docker history nginx:alpine

# Удаление образа
docker rmi nginx:alpine  # (для выполнеия следующих пунктов задания установить
снова после удаления!)
```

Практическое задание (выполнить самостоятельно!): Найдите образ PostgreSQL версии 15 и golang версии 1.21 с минимальным размером (alpine), скачайте его и сохраните вывод команды в отчете.

2.2 Жизненный цикл контейнеров

Задание 2.2.1: Запустите и управляйте контейнерами

```
# Запуск контейнера alpine в интерактивном режиме (при отсутствии образа на компьютере при выполнении этой команды должен загрузить автоматически!)

docker run -it --name test-alpine alpine:latest sh
# Внутри контейнера выполните: ls -la, exit. Должно отобразиться содержимое корневой папки контейнера alpine, затем exit выводит нас из интерактивного режима shell контейнера.

# Запуск контейнера в фоновом режиме
docker run -d --name web-server -p 8080:80 nginx:alpine

# Просмотр запущенных контейнеров
docker ps

# Просмотр всех контейнеров (включая остановленные)
docker ps -a

# Просмотр логов контейнера
docker logs web-server

# Подключение к работающему контейнеру
docker exec -it web-server sh
# Внутри контейнера выполните: cat /usr/share/nginx/html/index.html, exit

# Остановка контейнера
docker stop web-server

# Запуск остановленного контейнера
docker start web-server

# Удаление контейнера
docker rm web-server  # предварительно остановите
```

Практическое задание (выполнить самостоятельно!): Запустите контейнер с PostgreSQL, задав пароль через переменную окружения, подключитесь к нему с помощью `docker exec` и выполните SQL-запрос `SELECT version();` с помощью утилиты psql (как её использовать также изучить самомстоятельно!). Сохраните вывод в файл отчёта.

2.3 Работа с томами (Volumes)

Задание 2.3.1: Сохраните данные вне контейнера

Также нужно подставить своё ФИО и группу в месте, гже это указано.

```
# Создание именованного тома
docker volume create my-app-data

# Просмотр томов
docker volume ls
```

```
# Информация о томе
docker volume inspect my-app-data

# Запуск контейнера с томом
docker run -d `
  --name postgres-db `
  -e POSTGRES_PASSWORD=secret `
  -e POSTGRES_DB=testdb `
  -v my-app-data:/var/lib/postgresql/data `
  -p 5432:5432 `
  postgres:15-alpine

# Создание тестовой таблицы
docker exec -it postgres-db psql -U postgres -d testdb -c "CREATE TABLE users (id
SERIAL, name TEXT);"
docker exec -it postgres-db psql -U postgres -d testdb -c "INSERT INTO users
(name) VALUES ('Ваше ФИО (аббревиатура) и группа');"
Ф
# Остановка и удаление контейнера (данные сохранятся в томе)
docker stop postgres-db
docker rm postgres-db

# Запуск нового контейнера с тем же томом
docker run -d `
  --name new-postgres `
  -e POSTGRES_PASSWORD=secret `
  -e POSTGRES_DB=testdb `
  -v my-app-data:/var/lib/postgresql/data `
  -p 5432:5432 `
  postgres:15-alpine

# Проверка, что данные сохранились
docker exec -it new-postgres psql -U postgres -d testdb -c "SELECT * FROM users;"
```

Практическое задание (выполнить самостоятельно!): Создайте том для статических файлов, запустите Nginx с примонтированным томом, скопируйте в него файл `index.html` с помощью `docker cp` по пути внутри контейнера `/usr/share/nginx/html` и проверьте доступность страницы в браузере. Также нужно подставить своё ФИО и группу в месте, гже это указано.

Вот файл index.html

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Nginx Test Page</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 40px;
```

```
            background-color: #f4f4f4;
            text-align: center;
        }
        .container {
            background-color: white;
            border-radius: 10px;
            padding: 30px;
            max-width: 600px;
            margin: 0 auto;
            box-shadow: 0 2px 10px rgba(0,0,0,0.1);
        }
        h1 {
            color: #28a745;
        }
        .nginx-logo {
            font-size: 48px;
            margin: 20px 0;
            color: #009639;
        }
        .info {
            background-color: #e9ecef;
            border-radius: 5px;
            padding: 15px;
            margin: 20px 0;
            text-align: left;
        }
        .footer {
            margin-top: 30px;
            color: #666;
            font-size: 14px;
        }
    </style>
</head>
<body>
    <div class="container">
        <div class="nginx-logo">🌑</div>
        <h1>Ваше ФИО и группа</h1>

        <div class="info">
            <h3>Информация о сервере:</h3>
            <p><strong>Время на сервере:</strong> <span id="server-time"></span>
</p>
            <p><strong>IP-адрес сервера:</strong> <?php echo
$_SERVER['SERVER_ADDR']; ?></p>
            <p><strong>Имя сервера:</strong> <?php echo gethostname(); ?></p>
        </div>

        <div class="footer">
            <p>Тестовая страница Nginx • Создано для проверки конфигурации</p>
            <p>📍 /usr/share/nginx/html/index.html</p>
        </div>
    </div>

    <script>
```

```
        document.getElementById('server-time').textContent = new
Date().toLocaleString('ru-RU');
    </script>
</body>
</html>
```

2.4 Сеть в Docker

Задание 2.4.1: Создайте изолированную сеть для взаимодействия контейнеров

```
# Создание сети
docker network create my-app-network

# Просмотр сетей
docker network ls

# Запуск контейнеров в одной сети
docker run -d `
  --name postgres `
  --network my-app-network `
  -e POSTGRES_PASSWORD=secret `
  -e POSTGRES_DB=myapp `
  postgres:15-alpine

docker run -d `
  --name app `
  --network my-app-network `
  -p 8080:80 `
  nginx:alpine

# Проверка связи между контейнерами
docker exec app ping postgres  # должен быть ping
```

Практическое задание (выполнить самостоятельно!): Создайте сеть типа `bridge`, запустите в ней два контейнера (Nginx и PostgreSQL), проверьте их взаимодействие по именам контейнеров (можно сделать ping из контейнера к другому контейнеру через `docker exec`). Отразить вывод в отчет.

Часть 3. Разработка многокомпонентного приложения

3.1 Backend на Go

Создайте файл `app/main.go` со следующим содержимым:

```
package main

import (
    "database/sql"
```

```go
    "encoding/json"
    "fmt"
    "log"
    "net/http"
    "os"
    "strconv"
    "strings"

    _ "github.com/lib/pq"
)

type User struct {
    ID   int    `json:"id"`
    Name string `json:"name"`
}

var db *sql.DB

func main() {
    // Подключение к PostgreSQL
    connStr := fmt.Sprintf(
        "host=%s port=%s user=%s password=%s dbname=%s sslmode=disable",
        getEnv("DB_HOST", "localhost"),
        getEnv("DB_PORT", "5432"),
        getEnv("DB_USER", "postgres"),
        getEnv("DB_PASSWORD", "secret"),
        getEnv("DB_NAME", "myapp"),
    )

    var err error
    db, err = sql.Open("postgres", connStr)
    if err != nil {
        log.Fatal("Error connecting to database:", err)
    }
    defer db.Close()

    // Создание таблицы
    createTableSQL := `
    CREATE TABLE IF NOT EXISTS users (
        id SERIAL PRIMARY KEY,
        name TEXT NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );`

    _, err = db.Exec(createTableSQL)
    if err != nil {
        log.Fatal("Error creating table:", err)
    }

    // Маршруты
    http.HandleFunc("/", homeHandler)
    http.HandleFunc("/api/users", usersHandler)
    http.HandleFunc("/api/users/add", addUserHandler)
    http.HandleFunc("/api/health", healthHandler)
```

```go
    http.HandleFunc("/api/users/", singleUserHandler)
    port := getEnv("PORT", "8080")
    log.Printf("Server starting on port %s", port)
    log.Fatal(http.ListenAndServe(":"+port, nil))
}
func singleUserHandler(w http.ResponseWriter, r *http.Request) {

    pathParts := strings.Split(r.URL.Path, "/")
    if len(pathParts) < 3 {
        http.Error(w, "Invalid URL", http.StatusBadRequest)
        return
    }
    idStr := pathParts[len(pathParts)-1]
    id, err := strconv.Atoi(idStr)
    if err != nil {
        http.Error(w, "Invalid ID format", http.StatusBadRequest)
        return
    }

    db.Exec("DELETE FROM users WHERE id=$1", id)

    //  if err != nil {
    //      http.Error(w, err.Error(), http.StatusInternalServerError)
    //      return
    //  }

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(map[string]interface{}{
        "id":      id,
        "message": "User deleted successfully",
    })

}
func homeHandler(w http.ResponseWriter, r *http.Request) {
    http.ServeFile(w, r, "/static/index.html")
}

func usersHandler(w http.ResponseWriter, r *http.Request) {
    rows, err := db.Query("SELECT id, name FROM users ORDER BY id DESC")
    if err != nil {
        http.Error(w, err.Error(), http.StatusInternalServerError)
        return
    }
    defer rows.Close()

    var users []User
    for rows.Next() {
        var u User
        if err := rows.Scan(&u.ID, &u.Name); err != nil {
            http.Error(w, err.Error(), http.StatusInternalServerError)
            return
        }
        users = append(users, u)
    }
```

```
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(users)
}

func addUserHandler(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodPost {
        http.Error(w, "Method not allowed", http.StatusMethodNotAllowed)
        return
    }

    name := r.FormValue("name")
    if name == "" {
        http.Error(w, "Name is required", http.StatusBadRequest)
        return
    }

    var id int
    err := db.QueryRow("INSERT INTO users (name) VALUES ($1) RETURNING id",
name).Scan(&id)
    if err != nil {
        http.Error(w, err.Error(), http.StatusInternalServerError)
        return
    }

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(map[string]interface{}{
        "id":      id,
        "name":    name,
        "message": "User created successfully",
    })
}

func healthHandler(w http.ResponseWriter, r *http.Request) {
    if err := db.Ping(); err != nil {
        w.WriteHeader(http.StatusServiceUnavailable)
        json.NewEncoder(w).Encode(map[string]string{"status": "unhealthy"})
        return
    }
    w.WriteHeader(http.StatusOK)
    json.NewEncoder(w).Encode(map[string]string{"status": "healthy"})
}

func getEnv(key, defaultValue string) string {
    if value := os.Getenv(key); value != "" {
        return value
    }
    return defaultValue
}
```

Создайте файл `app/go.mod`:

```
module  containers-lab-1 //главное, чтобы название модуля совпадало с его
названием в Dockerfile ниже

go 1.21

require github.com/lib/pq v1.10.9
```

3.2 Frontend и статика

Создайте файл `nginx/index.html` (весь код тестового сайта порсто копируем!):

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Containerization Demo</title>
    <style>
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            max-width: 800px;
            margin: 0 auto;
            padding: 20px;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            color: white;
        }
        .container {
            background: rgba(255, 255, 255, 0.9);
            border-radius: 10px;
            padding: 30px;
            box-shadow: 0 20px 60px rgba(0,0,0,0.3);
            color: #333;
        }
        h1 {
            color: #667eea;
            margin-top: 0;
        }
        form {
            margin: 20px 0;
            display: flex;
            gap: 10px;
        }
        input[type="text"] {
            flex: 1;
            padding: 10px;
            border: 2px solid #ddd;
            border-radius: 5px;
            font-size: 16px;
        }
        button {
```

```
            background: #667eea;
            color: white;
            border: none;
            padding: 10px 20px;
            border-radius: 5px;
            cursor: pointer;
            font-size: 16px;
            transition: background 0.3s;
        }
        button:hover {
            background: #764ba2;
        }
        table {
            width: 100%;
            border-collapse: collapse;
            margin-top: 20px;
        }
        th, td {
            padding: 12px;
            text-align: left;
            border-bottom: 1px solid #ddd;
        }
        th {
            background: #667eea;
            color: white;
        }
        .badge {
            display: inline-block;
            padding: 5px 10px;
            background: #4CAF50;
            color: white;
            border-radius: 3px;
            font-size: 12px;
            margin-left: 10px;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>🐳 Docker Multi-Container Application</h1>
        <p>Это приложение демонстрирует работу с Docker контейнерами:
            <span class="badge">Go Backend</span>
            <span class="badge">PostgreSQL</span>
            <span class="badge">Nginx</span>
        </p>

        <h2>Добавить пользователя</h2>
        <form id="userForm">
            <input type="text" id="userName" placeholder="Введите имя" required>
            <button type="submit">Добавить</button>
        </form>

        <h2>Список пользователей</h2>
        <table id="usersTable">
```

```
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Имя</th>
                    <th>Действия</th>
                </tr>
            </thead>
            <tbody id="usersList"></tbody>
        </table>

        <div style="margin-top: 20px; padding: 10px; background: #f5f5f5; border-
radius: 5px;">
            <strong>Информация о контейнере:</strong>
            <pre id="containerInfo" style="margin: 10px 0 0; font-size: 12px;">
</pre>
        </div>
    </div>

    <script>
        async function loadUsers() {
            const response = await fetch('/api/users');
            const users = await response.json();
            const tbody = document.getElementById('usersList');
            tbody.innerHTML = '';
            users.forEach(user => {
                const row = tbody.insertRow();
                row.insertCell(0).textContent = user.id;
                row.insertCell(1).textContent = user.name;
                const deleteCell = row.insertCell(2);
                const deleteBtn = document.createElement('button');
                deleteBtn.textContent = 'Удалить';
                deleteBtn.style.background = '#f44336';
                deleteBtn.onclick = () => deleteUser(user.id);
                deleteCell.appendChild(deleteBtn);
            });
        }

        document.getElementById('userForm').addEventListener('submit', async (e)
=> {
            e.preventDefault();
            const name = document.getElementById('userName').value;
            const formData = new FormData();
            formData.append('name', name);

            await fetch('/api/users/add', {
                method: 'POST',
                body: formData
            });

            document.getElementById('userName').value = '';
            loadUsers();
        });

        async function deleteUser(id) {
```

```
            if (confirm('Удалить пользователя?')) {
                await fetch(`/api/users/${id}`, { method: 'DELETE' });
                loadUsers();
            }
        }

        async function getContainerInfo() {
            const response = await fetch('/api/health');
            const data = await response.json();
            document.getElementById('containerInfo').textContent =
                `Статус: ${data.status}\nХост: ${window.location.hostname}\nВремя:
${new Date().toLocaleString()}`;
        }

        loadUsers();
        getContainerInfo();
        setInterval(getContainerInfo, 5000);
    </script>
</body>
</html>
```

Создайте файл `nginx/nginx.conf`:

```
events {
    worker_connections 1024;
}

http {
    upstream backend {
        server app:8080;
    }

    server {
        listen 80;
        server_name localhost;

        location / {
            root /usr/share/nginx/html;
            index index.html;
            try_files $uri $uri/ /index.html;
        }

        location /api/ {
            proxy_pass http://backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }
    }
}
```

3.3 Скрипты инициализации БД

Создайте файл `init-scripts/init.sql`:

```sql
CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    name TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO users (name) VALUES
    ('Docker Beginner'),
    ('Container Master'),
    ('DevOps Student')
ON CONFLICT DO NOTHING;
```

Часть 4. Multi-stage сборка Docker-образов

4.1 Dockerfile для Go приложения

Создайте файл `app/Dockerfile`:

```dockerfile
# Stage 1: Build
FROM golang:1.21-alpine AS builder

# Установка зависимостей для сборки
RUN apk add --no-cache git

# Рабочая директория
WORKDIR /app

# Копирование файлов зависимостей
COPY go.mod  ./

# Загрузка зависимостей
RUN go mod download
RUN go get containers-lab-1
# Копирование исходного кода
COPY . .

# Сборка приложения
RUN CGO_ENABLED=0 GOOS=linux go build -a -installsuffix cgo -o main .

# Stage 2: Final
FROM alpine:latest

# Установка CA сертификатов для HTTPS
RUN apk --no-cache add ca-certificates
```

```
# Рабочая директория
WORKDIR /root/

# Копирование собранного приложения из builder
COPY --from=builder /app/main .

# Копирование статических файлов
COPY --from=builder /app/static ./static

# Экспонирование порта
EXPOSE 8080

# Запуск приложения
CMD ["./main"]
```

4.2 Docker-compose для локальной разработки

Создайте файл `docker-compose.yml` в корне проекта:

```
version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    container_name: lab1-postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: myapp
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./init-scripts:/docker-entrypoint-initdb.d
    ports:
      - "5432:5432"
    networks:
      - app-network
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5

  app:
    build:
      context: ./app
      dockerfile: Dockerfile
    container_name: lab1-app
    environment:
      DB_HOST: postgres
      DB_PORT: 5432
```

```
      DB_USER: postgres
      DB_PASSWORD: secret
      DB_NAME: myapp
      PORT: 8080
    depends_on:
      postgres:
        condition: service_healthy
    networks:
      - app-network
    healthcheck:
      test: ["CMD", "wget", "--spider", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 3

  nginx:
    image: nginx:alpine
    container_name: lab1-nginx
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
      - ./nginx/index.html:/usr/share/nginx/html/index.html:ro
    ports:
      - "80:80"
    depends_on:
      app:
        condition: service_healthy
    networks:
      - app-network

volumes:
  postgres_data:
    name: lab1-postgres-data

networks:
  app-network:
    name: lab1-network
    driver: bridge
```

Часть 5. GitHub Actions для автоматизации

5.1 Создание Personal Access Token

1. Перейдите в настройки GitHub: Settings → Developer settings → Personal access tokens → Tokens (classic)
2. Создайте новый токен с правами: `write:packages`, `read:packages`
3. Скопируйте токен (он больше не будет показан)

5.2 Настройка Secrets в репозитории

1. Перейдите в ваш репозиторий на GitHub

2. Settings → Secrets and variables → Actions
3. Добавьте новый секрет: `CR_PAT` со значением вашего токена

5.3 Создание GitHub Actions workflow

Создайте файл `.github/workflows/docker-build.yml`, который должен будет собрать образы и доавить их в Github Registry, чтобы затем осуществить из них сборку:

```yaml
name: Build and Push Docker Images

on:
  push:
    branches: [ main, master ]
    tags:
      - 'v*'
  pull_request:
    branches: [ main, master ]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}

jobs:
  build:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Log in to GitHub Container Registry
        if: github.event_name != 'pull_request'
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.CR_PAT }}

      - name: Extract metadata for Go app
        id: meta-go
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}/app
          tags: |
            type=raw,value=latest,enable={{is_default_branch}}
            type=ref,event=branch
```

```
            type=ref,event=pr
            type=semver,pattern={{version}}
            type=semver,pattern={{major}}.{{minor}}
            type=sha

      - name: Build and push Go app
        uses: docker/build-push-action@v5
        with:
          context: ./app
          push: ${{ github.event_name != 'pull_request' }}
          tags: ${{ steps.meta-go.outputs.tags }}
          labels: ${{ steps.meta-go.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

      - name: Extract metadata for Nginx
        id: meta-nginx
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}/nginx
          tags: |
            type=raw,value=latest,enable={{is_default_branch}}
            type=ref,event=branch
            type=sha

      - name: Build and push Nginx
        if: github.event_name != 'pull_request'
        uses: docker/build-push-action@v5
        with:
          context: ./nginx
          file: ./nginx/Dockerfile
          push: true
          tags: ${{ steps.meta-nginx.outputs.tags }}
          labels: ${{ steps.meta-nginx.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

      - name: Test the built images
        run: |
          docker compose -f docker-compose.test.yml up -d
          sleep 30
          curl -f http://localhost/health || exit 1
          docker compose down
```

* ВАЖНО! Для корректной сборки в Github Actions также необходимо создать файл `./nginx/Dockerfile`, и спомощью команд COPY поместить во внутрь образа файлы `index.html` и `nginx.conf`

5.4 Тестовый docker-compose для CI

Создайте файл `docker-compose.test.yml`:

```yaml
version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: myapp_test
    networks:
      - test-network

  app:
    image: ghcr.io/${{ github.repository }}/app:sha-${{ github.sha }}
    environment:
      DB_HOST: postgres
      DB_PASSWORD: secret
      DB_NAME: myapp_test
    depends_on:
      - postgres
    networks:
      - test-network

  nginx:
    image: ghcr.io/${{ github.repository }}/nginx:sha-${{ github.sha }}
    ports:
      - "80:80"
    depends_on:
      - app
    networks:
      - test-network

networks:
  test-network:
```

Часть 6. Отчетность и проверка

6.1 Требования к отчету

Создайте в корне репозитория файл `REPORT.md` со следующим содержанием:

```markdown
# Отчет по практической работе №1
## Студент: [ФИО]
## Группа: [номер группы]
## Дата выполнения: [дата]

### 1. Выполненные команды Docker

#### 1.1 Работа с образами
\`\`\`
```

```
[вставьте вывод docker images либо скриншоты]
\`\`\`

#### 1.2 Работа с контейнерами
\`\`\`
[вставьте вывод docker ps -a либо скриншоты]
\`\`\`

#### 1.3 Работа с томами
\`\`\`
[вставьте вывод docker volume ls либо скриншоты]
\`\`\`

### 2. Скриншоты работающего приложения

#### 2.1 Главная страница
![Главная страница](screenshots/main-page.png)

#### 2.2 Добавление пользователя
![Добавление пользователя](screenshots/add-user.png)

#### 2.3 Список пользователей в БД
![PostgreSQL](screenshots/postgres-data.png)

### 3. GitHub Actions

#### 3.1 Успешный запуск workflow
![GitHub Actions](screenshots/github-actions.png)

#### 3.2 Опубликованные образы в GHCR
![GHCR Packages](screenshots/ghcr-packages.png)

### 4. Выводы
[Опишите, что нового узнали, с какими трудностями столкнулись]
```

6.2 Создание скриншотов

Создайте директорию `screenshots` и добавьте скриншоты:

1. `main-page.png` - главная страница приложения
2. `add-user.png` - процесс добавления пользователя
3. `postgres-data.png` - данные в PostgreSQL (через `docker exec`)
4. `github-actions.png` - успешный запуск Actions
5. `ghcr-packages.png` - опубликованные пакеты в GitHub Packages

Названия файлов скриншотов даны примерные, главное обеспечить видимость файлов скриншотов для REPORT.md и указать правильные относительные пути файлов и правильный синтаксис markdown для вставки изображений, в данном примере все файлы помещаются в папку screenshots, можно и без неё, тогда скрины должны быть в корне репозитория в одном каталоге с REPORT.md, как показано в примере!

Отчет для СДО можно сформировать любым средством конвретирования markdown в pdf, например, расширением в VS Code Markdown PDF!

6.3 Финальный коммит

```
git add .
git commit -m "feat: complete lab work 1"
git push origin main
```

Критерии оценки

Критерий	Баллы	Проверка
Выполнение базовых команд Docker	1	Наличие выводов команд в отчете
Работа с томами и сохранение данных	2	Скриншот данных после перезапуска
Multi-stage Dockerfile для Go приложения	2	Код в репозитории
Docker-compose с тремя сервисами	2	Работоспособность локально, на скриншотах и в БД фигурируют ваши ФИО и группа
Отчет в виде markdown разметки	2	Правильно вставлены изображения и соблюдена структура отчета с указанием ФИО и группы
GitHub Actions успешно выполнен	3	Зеленая галочка в Actions
Образы опубликованы в GHCR	3	Наличие пакетов в репозитории
ИТОГО	**100**	

Приложение: Шпаргалка по основным командам Docker

Команда	Описание	Пример
`docker pull`	Скачать образ	`docker pull nginx:alpine`
`docker images`	Список образов	`docker images`
`docker rmi`	Удалить образ	`docker rmi nginx:alpine`
`docker run`	Запустить контейнер	`docker run -d -p 80:80 nginx`
`docker ps`	Список контейнеров	`docker ps -a`
`docker stop`	Остановить контейнер	`docker stop web-server`

Команда	Описание	Пример
`docker start`	Запустить остановленный	`docker start web-server`
`docker rm`	Удалить контейнер	`docker rm web-server`
`docker exec`	Выполнить команду	`docker exec -it web-server sh`
`docker logs`	Просмотр логов	`docker logs -f web-server`
`docker cp`	Копировать файлы	`docker cp file.txt cont:/path`
`docker volume`	Управление томами	`docker volume create my-vol`
`docker network`	Управление сетями	`docker network create my-net`
`docker build`	Собрать образ	`docker build -t my-app .`
`docker push`	Отправить образ	`docker push ghcr.io/user/app`
`docker compose`	Запустить композ	`docker compose up -d`

Ссылки:

Препарируем слои образов Docker: https://habr.com/ru/companies/otus/articles/859256/

Шпаргалка с командами Docker: https://habr.com/ru/companies/flant/articles/336654/

Как настроить Nginx в качестве балансировщика нагрузки:
https://habr.com/ru/companies/first/articles/683870/